Summarize

Chapter 11. Managing Distributed Workflows

Tuesday, February 15, 14:34



Austen bolted into Logan's office just after lunch. "I've been looking at the new architecture designs, and I want to help out. Do you need me to write up some ADRs or help with some spikes? I'd be happy to write up the ADR that states that we're only going to use choreography in the new architecture to keep things decoupled."

"Whoa, there, you maniac," said Logan. "Where did you hear that? What gives you that impression?"

"Well, I've been reading a lot about microservices, and everyone's advice seems to be to keep things highly decoupled. When I look at the patterns for communication, it seems that choreography is the most decoupled, so we should always use it, right?"

"*Always* is a tricky term in software architecture. I had a mentor who had a memorable perspective on this, who always said, *Never use absolutes when talking about architecture, except when talking about absolutes.* In other words, *never* say *never*. I can't think of many decisions in architecture where *always* or *never* applies."

"OK," said Austen. "So how *do* architects decide between the different communication patterns?"

As part of our ongoing analysis of the trade-offs associated with modern distributed architectures, we reach the *dynamic* part of quantum coupling, realizing many of the patterns we described and named in Chapter 2. In fact, even our named patterns only touch on the many permutations possible with modern architectures. Thus, an architect should understand the forces at work so that they can make a most objective trade-off analysis.

In Chapter 2, we identified three coupling forces when considering interaction models in distributed architectures: communication, consistency, and coordination, shown in Figure 11-1.

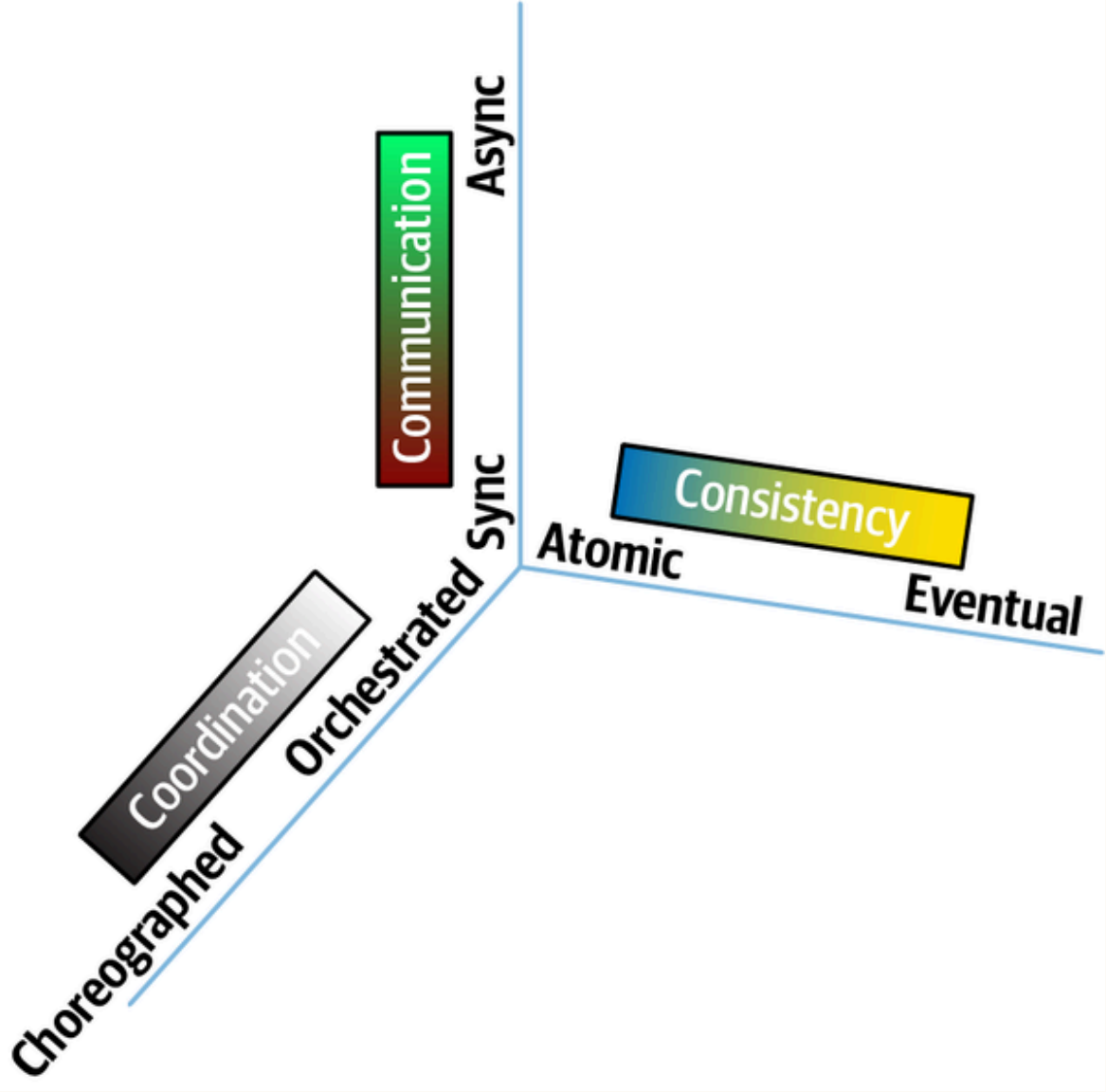


Figure 11-1. The dimensions of dynamic quantum coupling

In this chapter, we discuss *coordination*: combining two or more services in a distributed architecture to form some domain-specific work, along with the many attendant issues.

Two fundamental coordination patterns exist in distributed architectures: orchestration and choreography. The fundamental topological differences between the two styles is illustrated in Figure 11-2.

Orchestration is distinguished by the use of an orchestrator, whereas a choreographed solution does not use one.

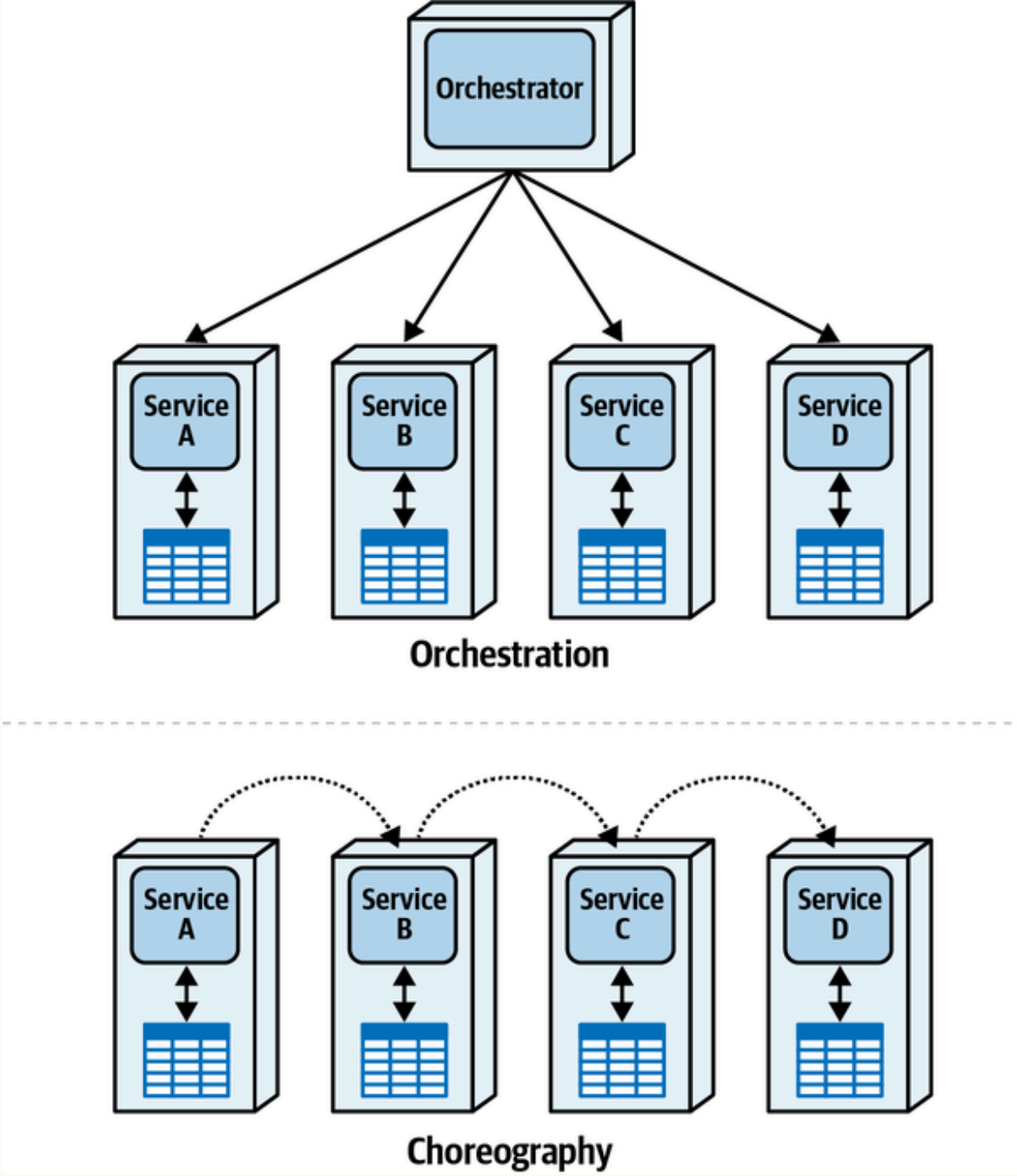


Figure 11-2. Orchestration versus choreography in distributed architectures

Orchestration Communication Style

The *orchestration pattern* uses an *orchestrator* (sometimes called a *mediator*) component to manage workflow state, optional behavior, error handling, notification, and a host of other workflow maintenance. It is named for the distinguishing feature of a musical orchestra, which utilizes a conductor to synchronize the incomplete parts of the overall score to create a unified piece of music. Orchestration is illustrated in the most generic representation in Figure 11-3.

In this example, services A-D are domain services, each responsible for its own bounded context, data, and behavior. The Orchestrator component generally doesn't include any domain behavior outside of the workflow it mediates. Notice that microservices architectures have an orchestrator per workflow, *not* a global orchestrator such as an *enterprise service*

bus (ESB). One of the primary goals of the microservices architecture style is decoupling, and using a global component such as an ESB creates an undesirable coupling point. Thus, microservices tend to have an orchestrator per workflow.

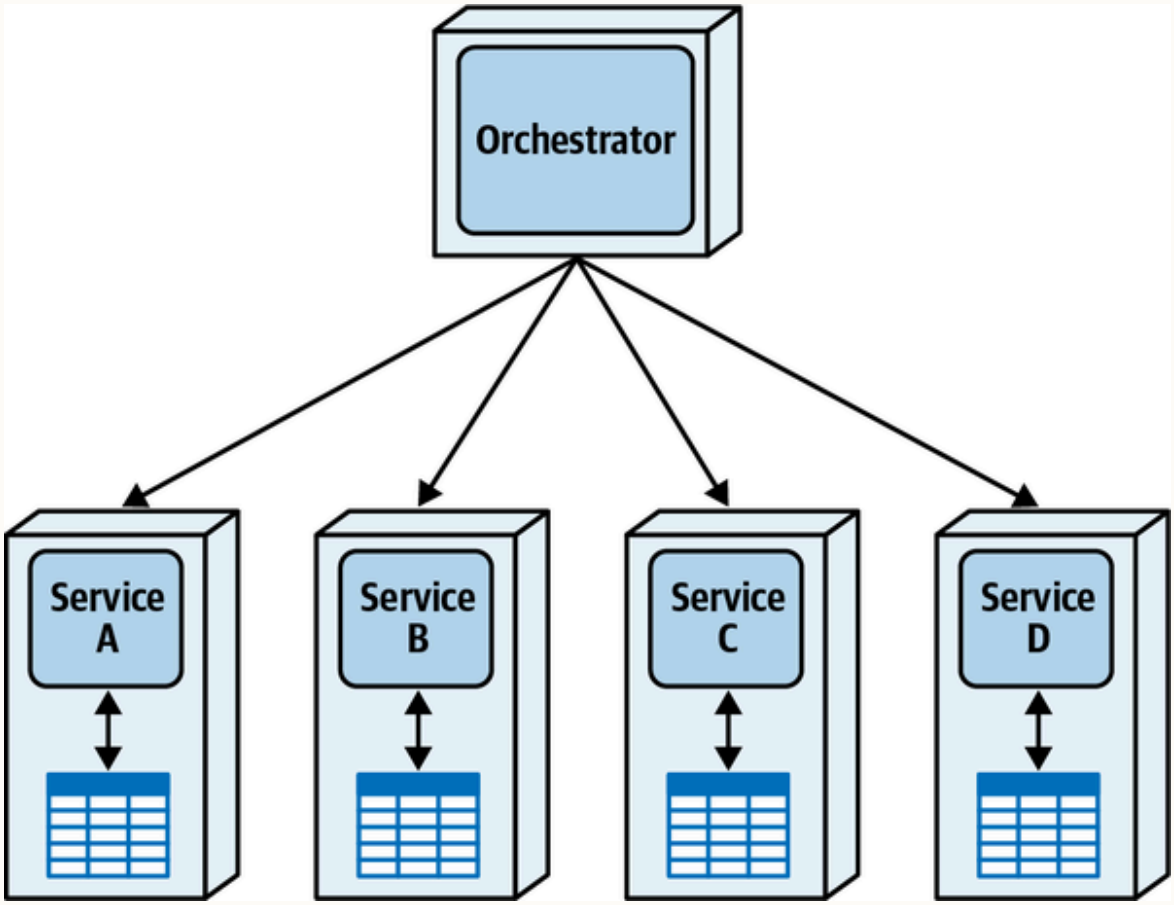


Figure 11-3. Orchestration among distributed microservices

Orchestration is useful when an architect must model a complex workflow that includes more than just the single "happy path," but also alternate paths and error conditions. However, to understand the basic shape of the pattern, we start with the nonerror happy path. Consider a very simple example of Penultimate Electronics selling a device to one of its customers online, shown in Figure 11-4.

This system passes the Place Order request to the Order Placement Orchestrator, which makes a synchronous call to the Order Placement Service, which records the order and returns a status message. Next, the mediator calls the Payment Service, which updates payment information. Next, the orchestrator makes an asynchronous call to the Fulfillment Service to handle the order. The call is asynchronous because no strict timing dependencies exist for order fulfillment, unlike payment verification. For example, if order fulfillment happens only a few times a day, there is no reason for the overhead of a synchronous call. Similarly, the orchestrator then calls the Email Service to notify the user of a successful electronics order.

If the world consisted of only happy paths, software architecture would be easy. However, one of the primary hard parts of software architecture is error conditions and pathways.

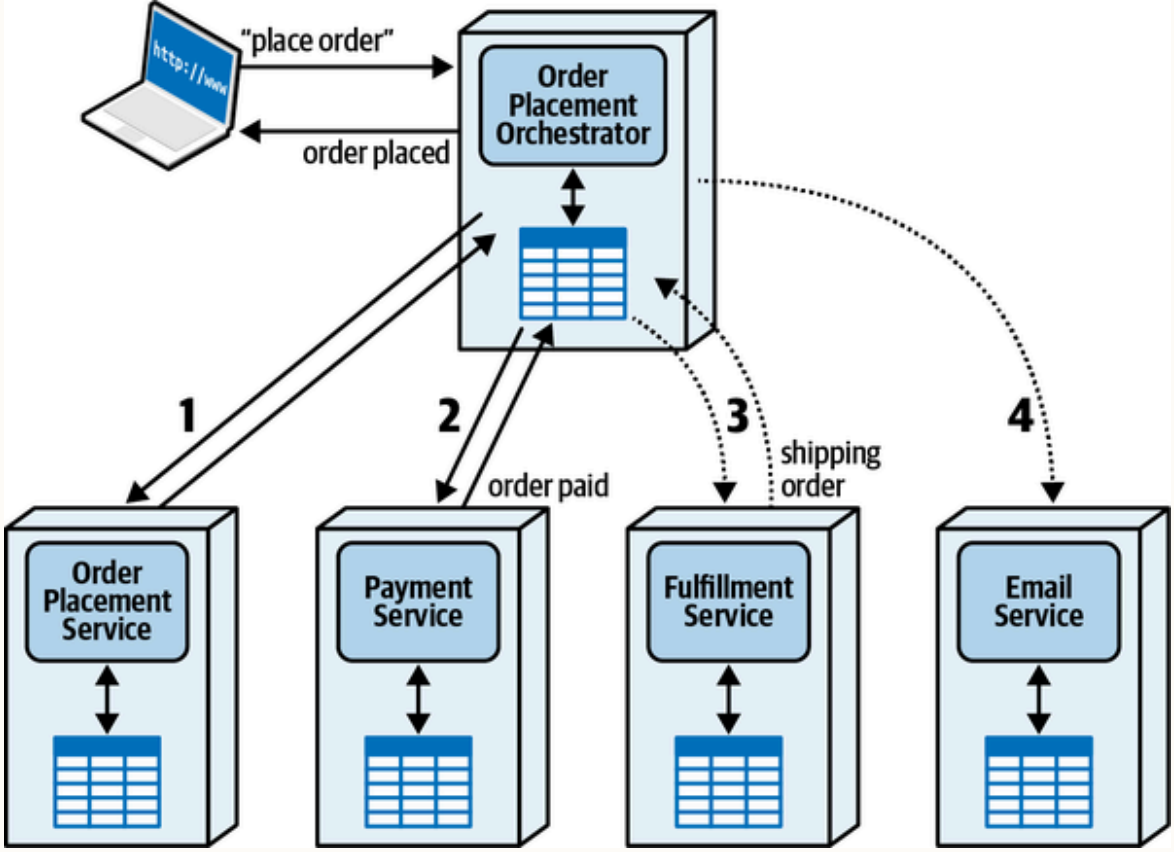


Figure 11-4. A "happy path" workflow using an orchestrator to purchase electronic equipment (note the asynchronous calls denoted by dotted lines for less time-sensitive calls)

Consider two potential error scenarios for electronics purchasing. First, what happens if the customer's payment method is rejected? This error scenario appears in Figure 11-5.

Here, the Order Placement Orchestrator updates the order via the Order Placement Service as before. However, when trying to apply payment, it is rejected by the payment service, perhaps because of an expired credit card number. In that case, the Payment Service notifies the orchestrator, which then places a (typically) asynchronous call to send a message to the Email Service to notify the customer of the failed order. Additionally, the orchestrator updates the state of the Order Placement Service, which still thinks this is an active order.

Notice in this example we're allowing each service to maintain its own transactional state, modeling our "Fairy Tale Saga(seo) Pattern". One of the hardest parts of modern architectures is managing transactions, which we cover in Chapter 12.

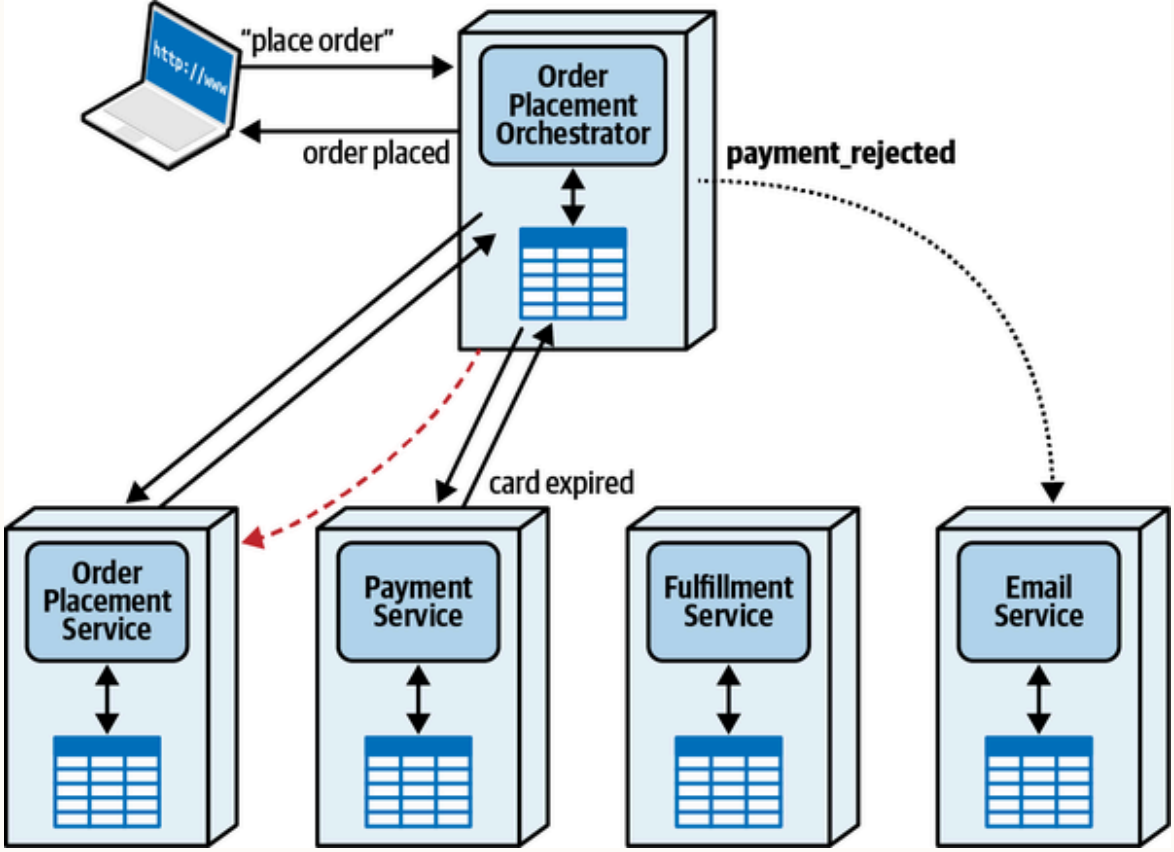


Figure 11-5. Payment rejected error condition

In the second error scenario, the workflow has progressed further along: what happens when the Fulfillment Service reports a back order? This error scenario appears in Figure 11-6.

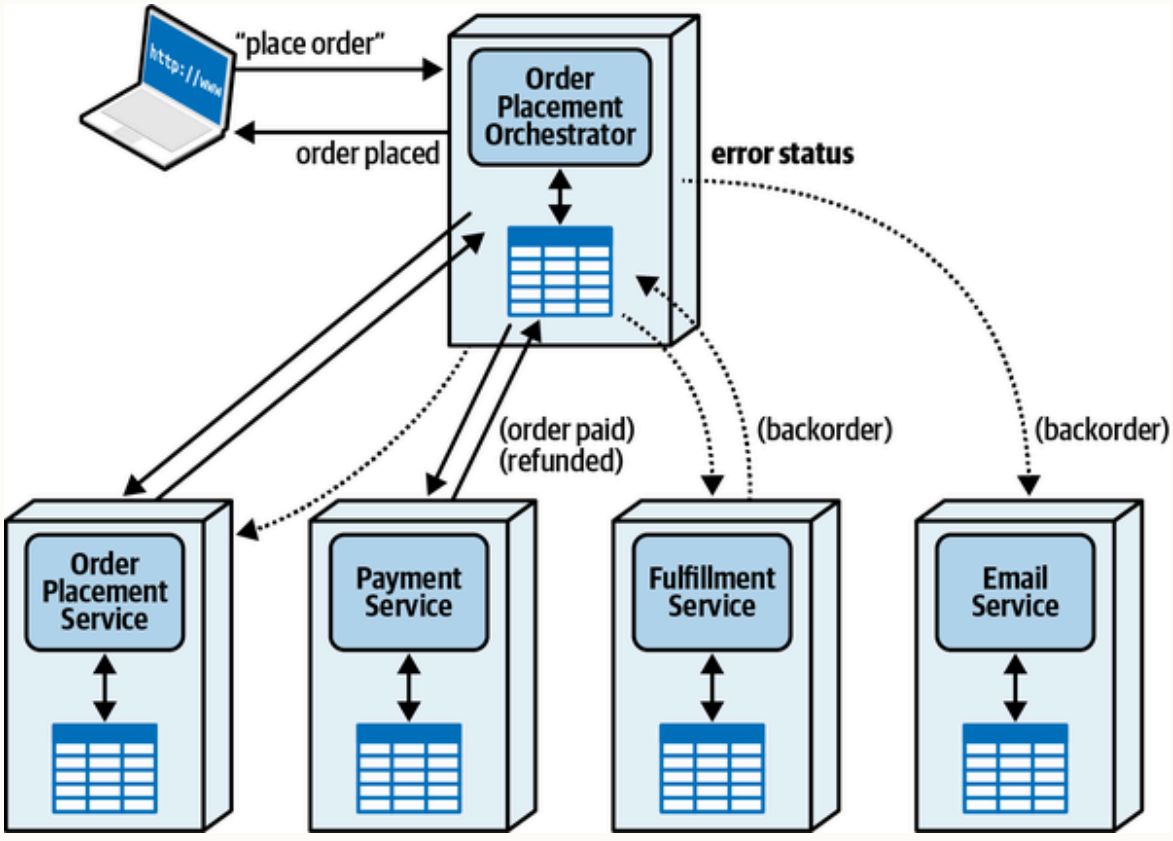


Figure 11-6. When an item is back-ordered, the orchestrator must rectify the state

As you can see, the workflow preceeds as normal until the Fulfillment Service notifies the orchestrator that the current item is out of stock, necessitating a back order. In that case,

the orchestrator must refund the payment (this is why many online services don't charge until shipment, not at order time) and update the state of the Order Placement Service.

One interesting characteristic to note in Figure 11-6: even in the most elaborate error scenarios, the architect wasn't required to add additional communication paths that weren't already there to facilitate the normal workflow, which differs from the "Choreography Communication Style".

General advantages of the orchestration communication style include the following:

Centralized workflow

As complexity goes up, having a unified component for state and behavior becomes beneficial.

Error handling

Error handling is a major part of many domain workflows, assisted by having a state owner for the workflow.

Recoverability

Because an orchestrator monitors the state of the workflow, an architect may add logic to retry if one or more domain services suffers from a short-term outage.

State management

Having an orchestrator makes the state of the workflow queriable, providing a place for other workflows and other transient states.

General disadvantages of the orchestration communication style include the following:

Responsiveness

All communication must go through the mediator, creating a potential throughput bottleneck that can harm responsiveness.

Fault tolerance

While orchestration enhances recoverability for domain services, it creates a potential single point of failure for the workflow, which can be addressed with redundancy but adds more complexity.

Scalability

This communication style doesn't scale as well as choreography because it has more coordination points (the orchestrator), which cuts down on potential parallelism. As

we discussed in Chapter 2, several dynamic coupling patterns utilize choreography and thus achieve higher scale (notably "Time Travel Saga(sec) Pattern" and "Anthology Saga(aec) Pattern").

Service coupling

Having a central orchestrator creates higher coupling between it and domain components, which is sometimes necessary. The orchestration communication style's trade-offs appear in Table 11-1.

TRADE-OFFS

Table 11-1. Trade-offs for orchestration

Advantage	Disadvantage
Centralized workflow	Responsiveness
Error handling	Fault tolerance
Recoverability	Scalability
State management	Service coupling

Choreography Communication Style

Whereas the Orchestration Communication Style was named for the metaphorical central coordination offered by an orchestrator, the *choreography* pattern visually illustrates intent of the communication style that has no central coordination. Instead, each service participates with the others, similar to dance partners. It isn't an ad hoc performance—the moves were planned beforehand by the choreographer/architect but executed without a central coordinator.

Figure 11-4 described the orchestrated workflow for a customer purchasing electronics from Penultimate Electronics; the same workflow modeled in the choreography communication style appears in Figure 11-7.

In this workflow, the initiating request goes to the first service in the chain of responsibility—in this case, the Order Placement Service. Once it has updated internal records about the order, it sends an asynchronous request that the Payment Service receives. Once payment

has been applied, the Payment Service generates a message received by the Fulfillment Service, which plans for delivery and sends a message to the Email Service.

At first glance, the choreography solution seems simpler—fewer services (no orchestrator), and a simple chain of events/commands (messages). However, as with many issues in software architecture, the difficulties lie not with the default paths but rather with boundary and error conditions.

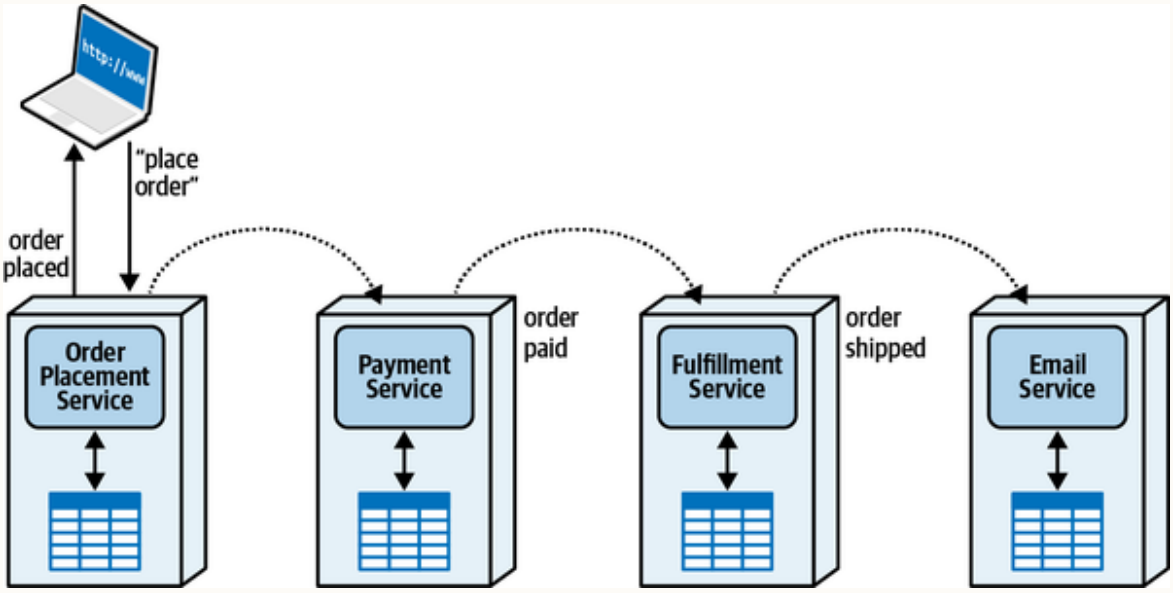


Figure 11-7. Purchasing electronics using choreography

As in the previous section, we cover two potential error scenarios. The first results from failed payment, as illustrated in Figure 11-8.

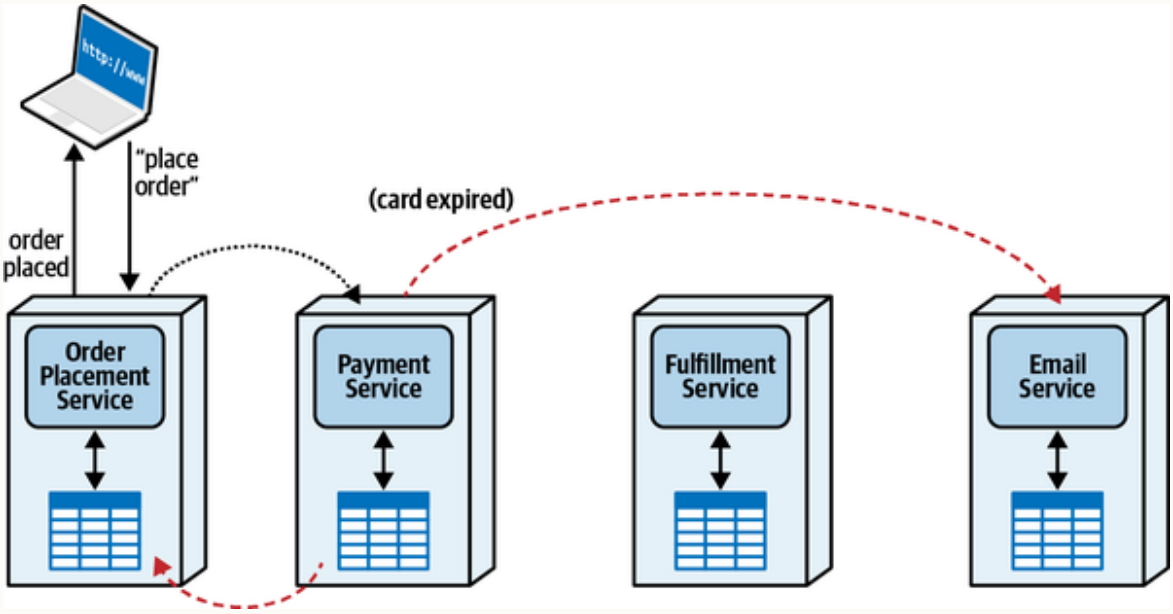


Figure 11-8. Error in payment in choreography

Rather than send a message intended for the Fulfillment Service, the Payment service sends messages indicating failure to the Email Service and back to the Order Placement Service to update the order status. This alternate workflow doesn't appear too complex, with a single new communication link that didn't exist before.

However, consider the increasing complexity imposed by the other error scenario for a product back order, shown in Figure 11-9.

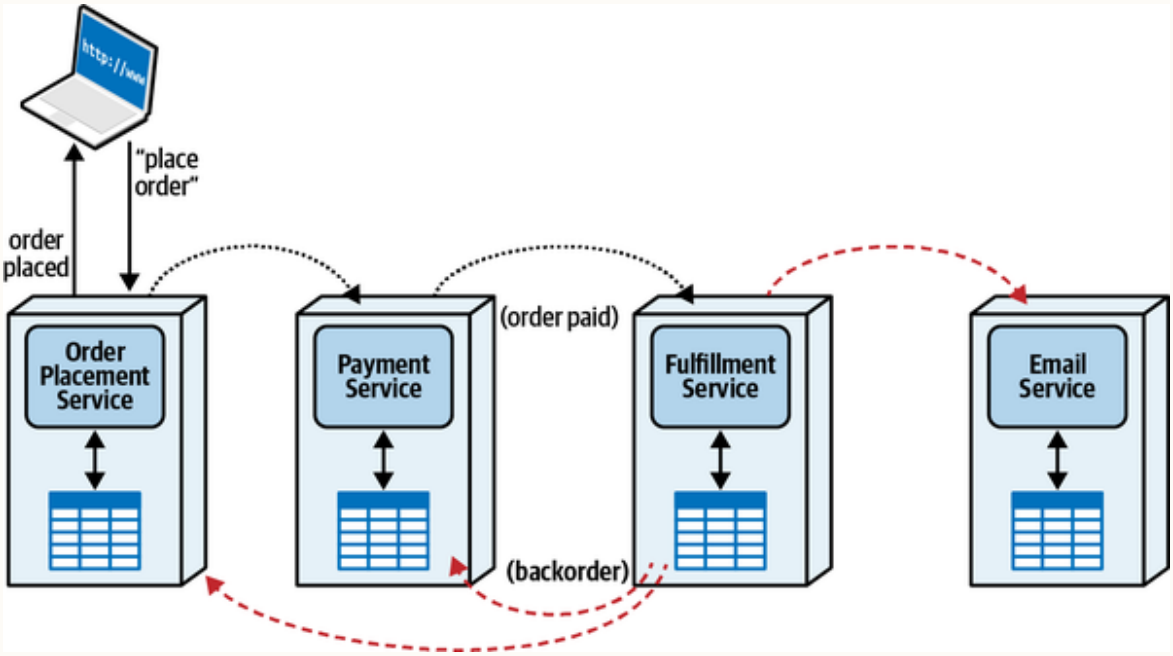


Figure 11-9. Managing the workflow error condition of product backlog

Many steps of this workflow have already completed before the event (out of stock) that causes the error. Because each of these services implement its own transactionality (this is an example of the "Anthology Saga(aec) Pattern"), when an error occurs, each service must issue compensating messages to other services. Once the Fulfillment Service realizes the error condition, it should generate events suited to its bounded context, perhaps a broadcast message subscribed to by the Email, Payment, and Order Placement services.

The example shown in Figure 11-9 illustrates the dependency between complex workflows and mediators. While the initial workflow in choreography illustrated in Figure 11-7 seemed simpler than Figure 11-4, the error case (and others) keeps adding more complexity to the choreographed solution. In Figure 11-10, each error scenario forces domain services to interact with each other, adding communication links that weren't necessary for the happy path.

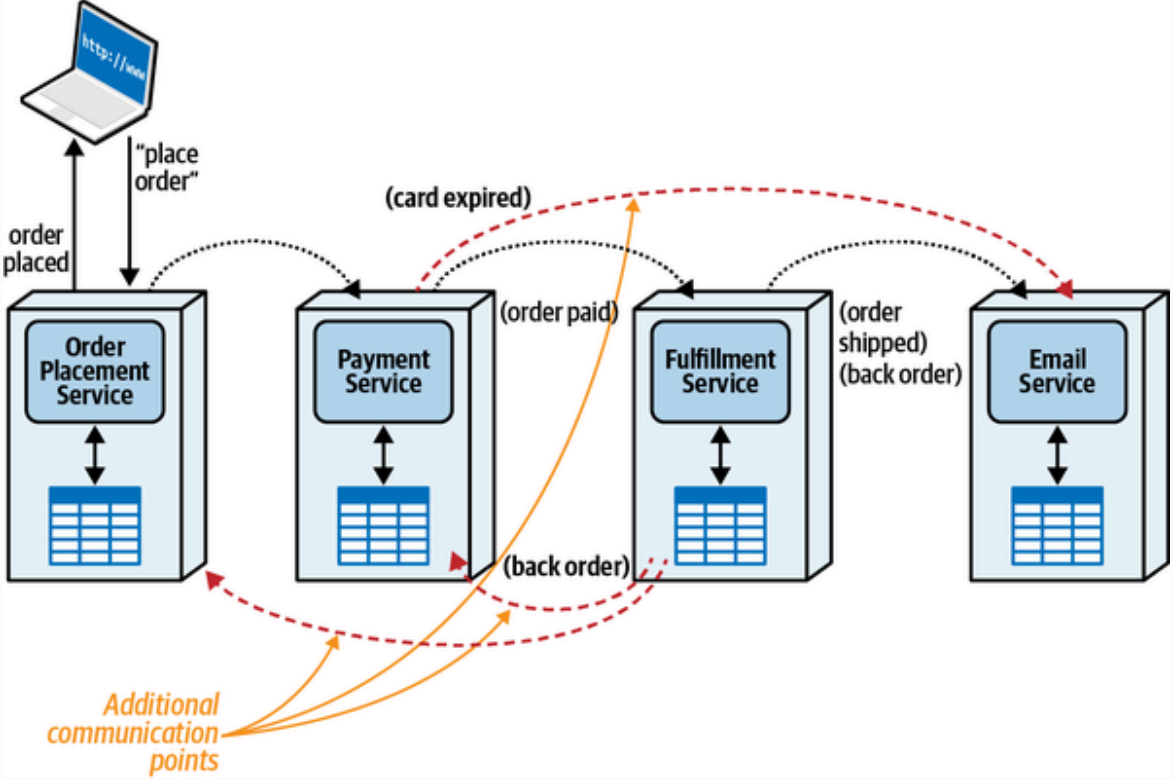


Figure 11-10. Error conditions in choreography typically add communication links

Every workflow that architects need to model in software has a certain amount of *semantic coupling*—the inherent coupling that exists in the problem domain. For example, the process of assigning a ticket to a Sysops Squad member has a certain workflow: a client must request service, skills must be matched to particular specialists, then cross-referenced to schedules and locations. The way an architect models that interaction is the *implementation coupling*.

The semantic coupling of a workflow is mandated by the domain requirements of the solution and must be modeled somehow. However clever an architect is, they cannot reduce the amount of semantic coupling, but their implementation choices may increase it. This doesn't mean that an architect might not push back on impractical or impossible semantics defined by business users—some domain requirements create extraordinarily difficult problems in architecture.

Here is a common example. Consider the standard layered monolithic architecture compared to the more modern style of a modular monolith, shown in Figure 11-11.

The architecture on the left represents the traditional layered architecture, separated by *technical* capabilities such as persistence, business rules, and so on. On the right, the same solution appears, but separated by *domain* concerns such as `Catalog Checkout` and `Update Inventory` rather than technical capabilities.

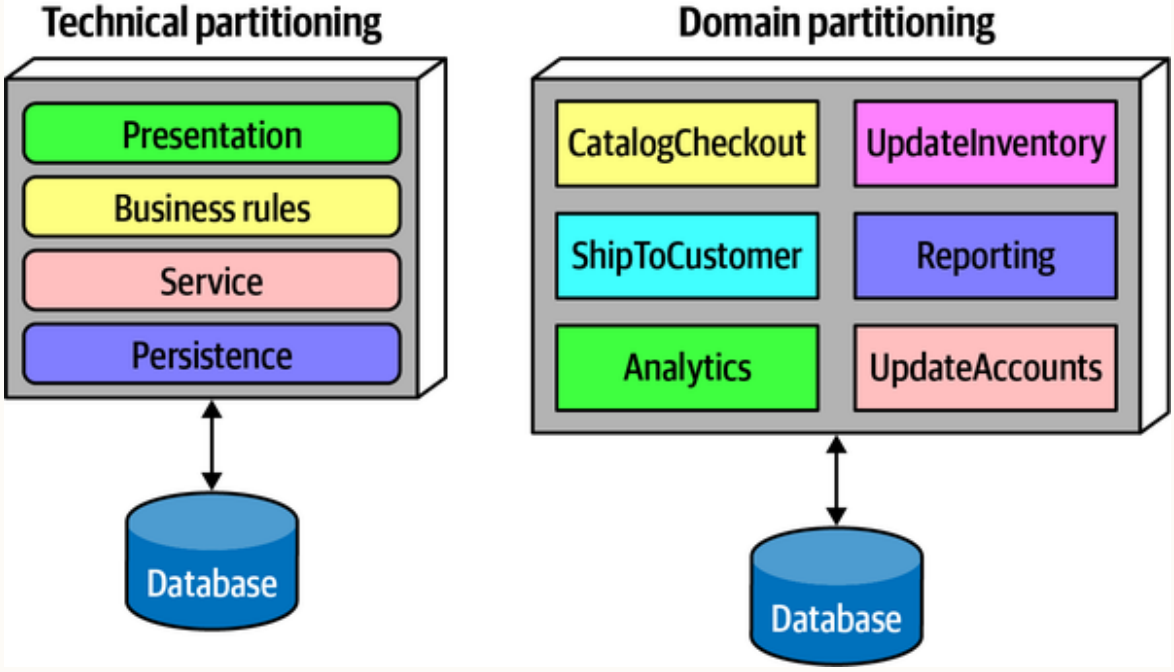


Figure 11-11. Technical versus domain partitioning in architecture

Both topologies are logical ways to organize a codebase. However, consider where domain concepts such as `Catalog Checkout` reside within each architecture, illustrated in Figure 11-12.

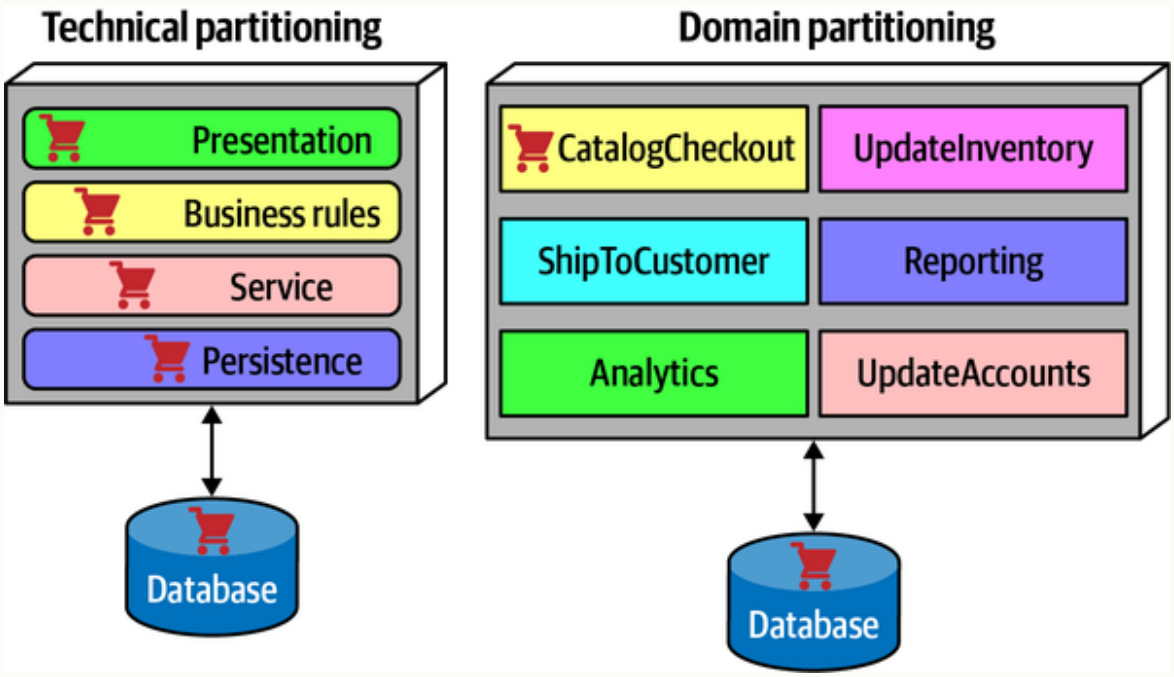


Figure 11-12. Catalog Checkout is smeared across implementation layers in a technically partitioned architecture

`Catalog Checkout` is "smeared" across the layers of the technical architecture, whereas it appears only in the matching domain component and database in the domain partitioned example. Of course, aligning a domain with domain partitioned architecture isn't a revelation—one of the insights of domain-driven design was the primacy of the domain workflows. No matter what, if an architect wants to model a workflow, they must make those moving parts work together. If the architect has organized their architecture the same as the domains, the implementation of the workflow should have similar complexity. However, if the architect has imposed additional layers (as in technical partitioning, shown

in Figure 11-12), it increases the overall implementation complexity because now the architect must design for the *semantic* complexity along with the additional *implementation* complexity.

Sometimes the extra complexity is warranted. For example, many layered architectures came from a desire by architects to gain cost savings by consolidating on architecture patterns, such as database connection pooling. In that case, an architect considered the trade-offs of the cost saving associated with technically partitioning database connectivity versus the imposed complexity and cost won in many cases.

The major lesson of the last decade of architecture design is to model the *semantics* of the workflow as closely as possible with the implementation.

TIP

An architect can never reduce semantic coupling via implementation, but they can make it worse.

Thus, we can establish a relationship between the semantic coupling and the need for coordination—the more steps required by the workflow, the more potential error and other optional paths appear.

Workflow State Management

Most workflows include transient state about the status of the workflow: what elements have executed, which ones are left, ordering, error conditions, retries, and so on. For orchestrated solutions, the obvious workflow state owner is the orchestrator (although some architectural solutions create stateless orchestrators for higher scale). However, for choreography, no obvious owner for workflow state exists. Many common options exist to manage state in choreography; here are three common ones.

First, the *Front Controller pattern* places the responsibility for state on the first called service in the chain of responsibility, which in this case is Order Placement Service. If that service contains information about both orders and the state of the workflow, some of the domain services must have a communication link to query and update the order state, as illustrated in Figure 11-13.

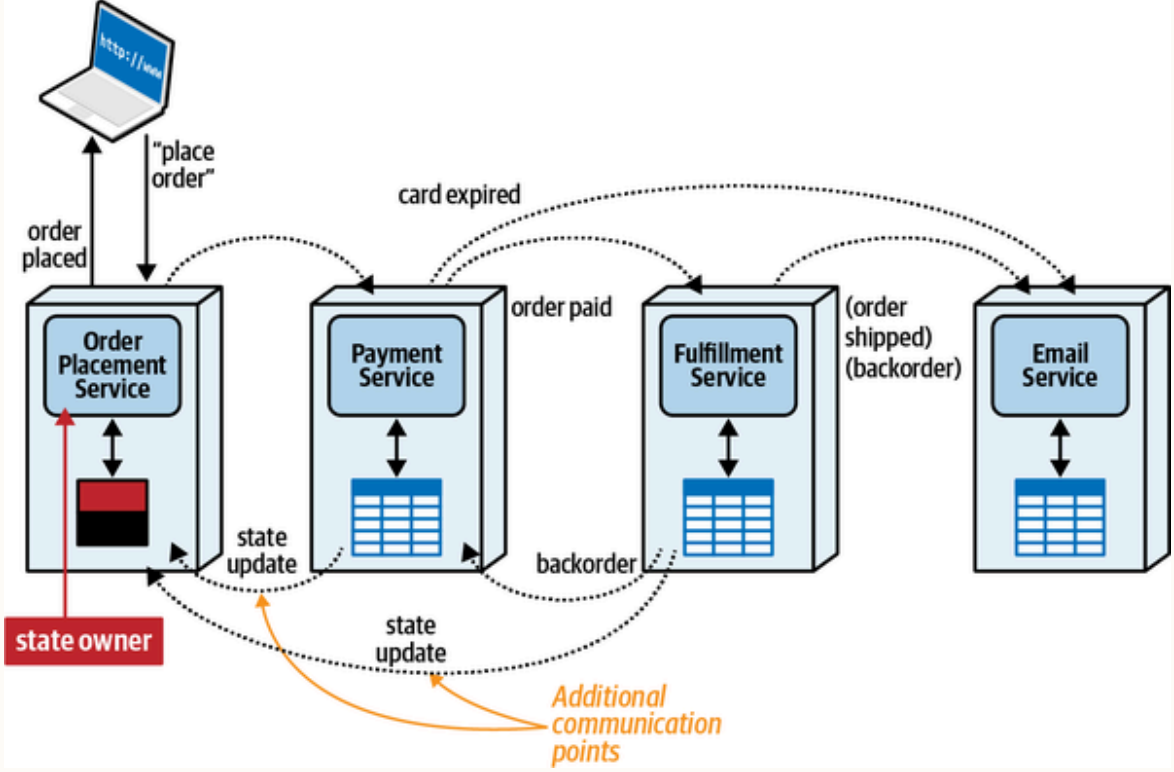


Figure 11-13. In choreography, a *Front Controller* is a domain service that owns workflow state in addition to domain behavior

In this scenario, some services must communicate back to the Order Placement Service to update the state of the order, as it is the state owner. While this simplifies the workflow, it increases communication overhead and makes the Order Placement Service more complex than one that handled only domain behavior. While the Front Controller pattern has some advantageous characteristics, it also has trade-offs, as shown in Table 11-2.

TRADE-OFFS

Table 11-2. Trade-offs for the Front Controller pattern

Advantage	Disadvantage
Creates a pseudo-orchestrator within choreography	Adds additional workflow state to a domain service
Makes querying the state of an order trivial	Increases communication overhead
	Detrimental to performance and scale as it increases integration communication chatter

---

A second way for an architect to manage the transactional state is to keep no transient workflow state at all, relying on querying the individual services to build a real-time snap-

shot. This is known as *stateless choreography*. While this simplifies the state of the first service, it greatly increases network overhead in terms of chatter between services to build a stateful snapshot. For example, consider a workflow like the simple choreography happy path in Figure 11-7 with no extra state. If a customer wants to know the state of their order, the architect must build a workflow that queries the state of each domain service to determine the most up-to-date order status. While this makes for a highly flexible solution, rebuilding state can be complex and costly in terms of operational architecture characteristics like scalability and performance. Stateless choreography trades high performance for workflow control, as illustrated in Table 11-3.

TRADE-OFFS

Table 11-3. Stateless choreography trade-offs

Advantage	Disadvantage
Offers high performance and scale	Workflow state must be built on the fly
Extremely decoupled	Complexity rises swiftly with complex workflows

---

A third solution utilizes *stamp coupling* (described in more detail in "Stamp Coupling for Workflow Management"), storing extra workflow state in the message contract sent between services. Each domain service updates its part of the overall state and passes that to the next in the chain of responsibility. Thus, any consumer of that contract can check on the status of the workflow without querying each service.

This is a partial solution, as it still does not provide a single place for users to query the state of the ongoing workflow. However, it does provide a way to pass the state between services as part of the workflow, providing each service with additional potentially useful context. As in all features of software architecture, stamp coupling has good and bad characteristics, shown in Table 11-4.

TRADE-OFFS

Table 11-4. Stamp coupling trade-offs

Advantage	Disadvantage
Allows domain services to pass workflow state without additional queries to a state owner	Contracts must be larger to accommodate workflow state
Eliminates need for a front controller	Doesn't provide just-in-time status queries

In Chapter 13, we discuss how contracts can reduce or increase workflow coupling in choreographed solutions.

Advantages of the choreography communication style include the following:

Responsiveness

This communication style has fewer single choke points, thus offering more opportunities for parallelism.

Scalability

Similar to responsiveness, lack of coordination points like orchestrators allows more independent scaling.

Fault tolerance

The lack of a single orchestrator allows an architect to enhance fault tolerance with the use of multiple instances.

Service decoupling

No orchestrator means less coupling.

Disadvantages of the choreography communication style include the following:

Distributed workflow

No workflow owner makes error management and other boundary conditions more difficult.

State management

No centralized state holder hinders ongoing state management.

Error handling

Error handling becomes more difficult without an orchestrator because the domain services must have more workflow knowledge.

Recoverability

Similarly, recoverability becomes more difficult without an orchestrator to attempt retries and other remediation efforts.

Like "Orchestration Communication Style", choreography has a number of good and bad trade-offs, often opposites, summarized in Table 11-5.

TRADE-OFFS

Table 11-5. Trade-offs for the choreography communication style

Advantage	Disadvantage
Responsiveness	Distributed workflow
Scalability	State management
Fault tolerance	Error handling
Service decoupling	Recoverability

Trade-Offs Between Orchestration and Choreography

As with all things in software architecture, neither orchestration nor choreography represent the perfect solution for all possibilities. A number of key trade-offs, including some delineated here, will lead an architect toward one of these two solutions.

State Owner and Coupling

As illustrated in Figure 11-13, state ownership typically resides somewhere, either in a formal mediator acting as an orchestrator, or a front controller in a choreographed solution. In

the choreographed solution, removing the mediator forces higher levels of communication between services. This might be a perfectly suitable trade-off. For example, if an architect has a workflow that needs higher scale and typically has few error conditions, it might be worth trading the higher scale of choreography with the complexity of error handling.

However, as workflow complexity goes up, the need for an orchestrator rises proportionally, as illustrated in Figure 11-14.

In addition, the more semantic complexity contained in a workflow, the more utilitarian an orchestrator is. Remember, implementation coupling can't make semantic coupling better, only worse.

Ultimately, the sweet spot for choreography lies with workflows that need responsiveness and scalability, and either don't have complex error scenarios or they are infrequent. This communication style allows for high throughput; it is used by the dynamic coupling patterns "Phone Tag Saga(sac) Pattern", "Time Travel Saga(sec) Pattern", and "Anthology Saga(aec) Pattern". However, it can also lead to extremely difficult implementations when other forces are mixed in, leading to the "Horror Story(aac) Pattern".

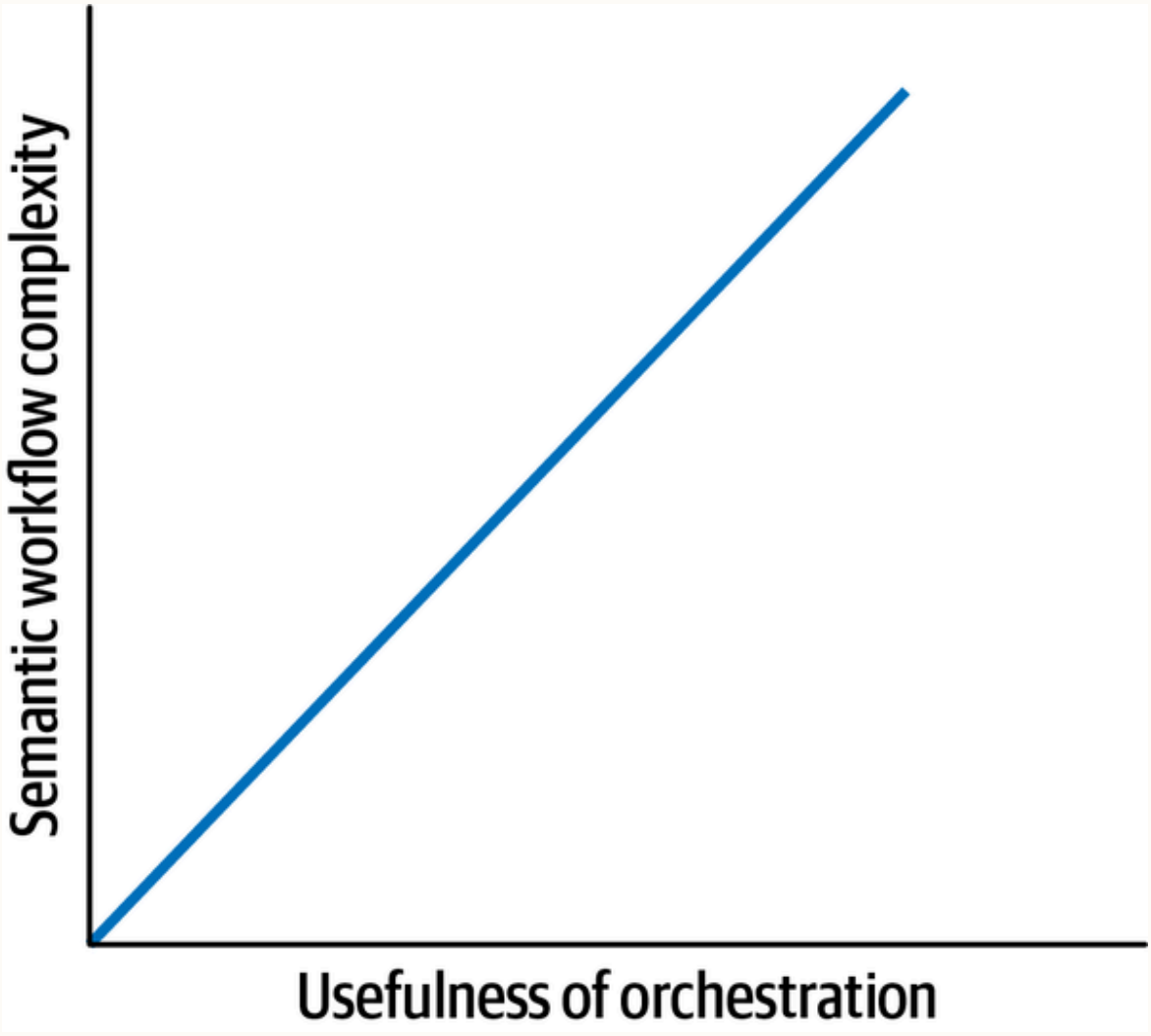


Figure 11-14. As the complexity of the workflow rises, orchestration becomes more useful

On the other hand, *orchestration* is best suited for complex workflows that include boundary and error conditions. While this style doesn't provide as much scale as choreography, it greatly reduces complexity in most cases. This communication style appears in "Epic Saga(sao) Pattern", "Fairy Tale Saga(seo) Pattern", "Fantasy Fiction Saga(aao) Pattern", and "Parallel Saga(aeo) Pattern".

Coordination is one of the primary forces that create complexity for architects when determining how to best communicate between microservices. Next, we investigate how this force intersects with another primary force, *consistency*.

Sysops Squad Saga: Managing Workflows

Thursday, March 15, 11:00



Addison and Austen arrived at Logan's office right on time, armed with a presentation and ritual coffee urn from the kitchen.

"Are you ready for us?" asked Addison.

"Sure," said Logan. "Good timing—just got off a conference call. Are y'all ready to talk about workflow options for the primary ticket flow?"

"Yes!" said Austen. "I think we should use choreography, but Addison thinks orchestration, and we can't decide."

"Give me an overview of the workflow we're looking at."

"It's the primary ticket workflow," said Addison. "It involves four services; here are the steps."

Customer-facing operations

1. Customer submits a trouble ticket through the Ticket Management Service and receives a ticket number.

Background operations

1. The Ticket Assignment Service finds the right Sysops expert for the trouble ticket.
2. The Ticket Assignment Service routes the trouble ticket to the systems expert's mobile device.

3. The customer is notified via the Notification Service that the Sysops expert is on their way to fix the problem.
4. The expert fixes the problem and marks the ticket as complete, which is sent to the Ticket Management Service.
5. The Ticket Management Service communicates with the Survey Service to tell the customer to fill out the survey.

"Have you modeled both solutions?" asked Logan.

"Yes. The drawing for choreography is in Figure 11-15."

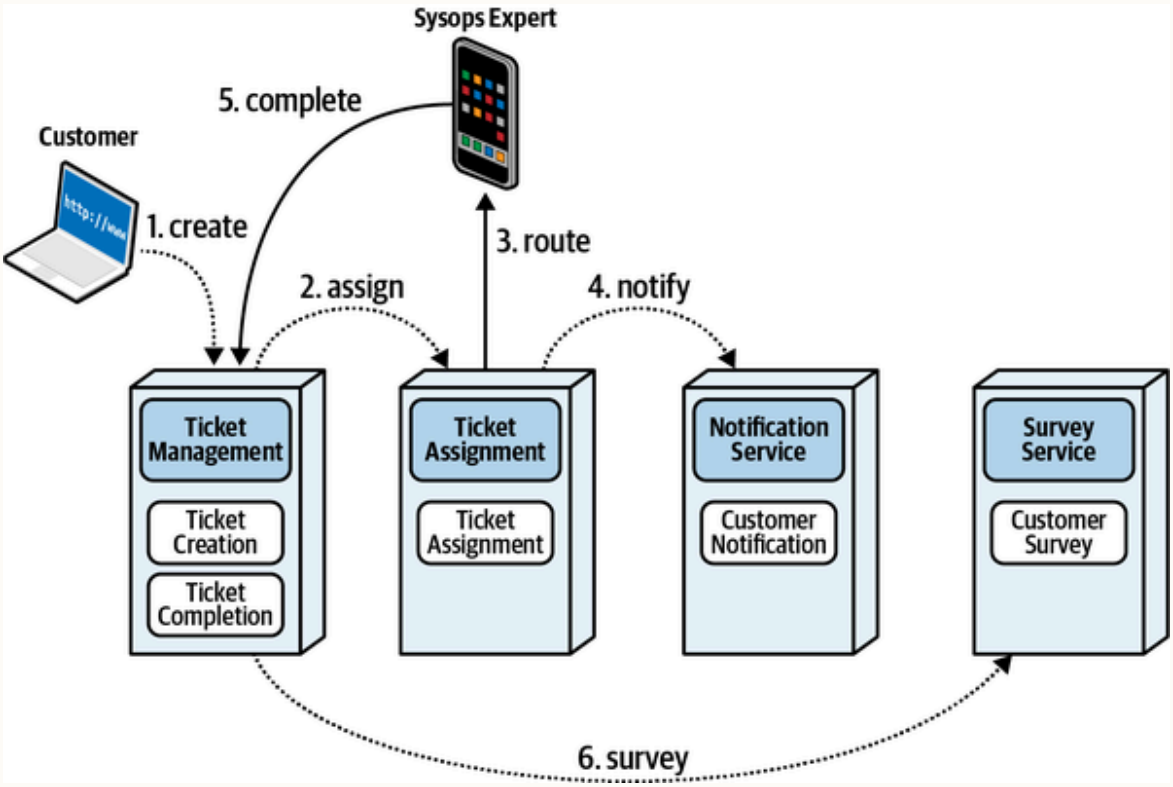


Figure 11-15. Primary ticket flow modeled as choreography

"...and the model for orchestration is in Figure 11-16."

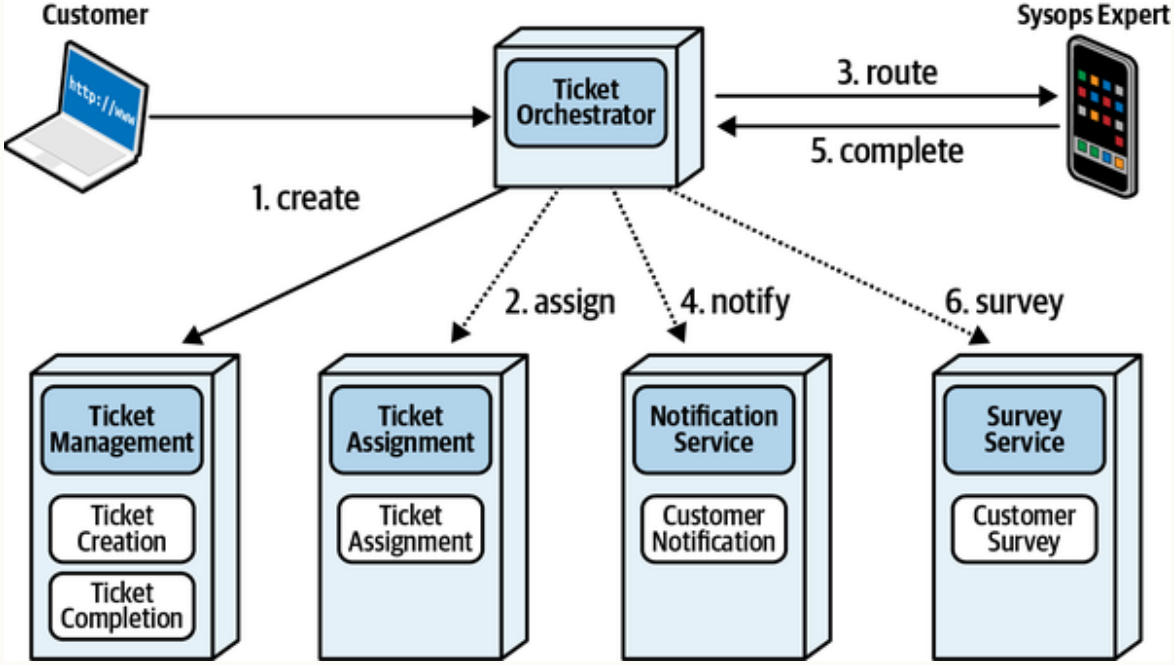


Figure 11-16. Primary ticket workflow modeled as orchestration

Logan pondered the diagrams for a moment, then pronounced, "Well, there doesn't seem to be an obvious winner here. You know what that means."

Austen piped up, "Trade-offs!"

"Of course," laughed Logan. " Let's think about the likely scenarios and see how each solution reacts to them. What are the primary issues you are concerned with?"

"The first is lost or misrouted tickets. The business has been complaining about it, and it has become a priority," said Addison.

"OK, which handles that problem better—orchestration or choreography?"

"Easier control of the workflow sounds like the orchestrator version is better—we can handle all the workflow issues there," volunteered Austen.

"OK, let's build a table of issues and preferred solutions in Table 11-6."

TRADE-OFFS

Table 11-6. Trade-off between orchestration and choreography for ticket workflow

Orchestration	Choreography
Workflow control	

"What's the next issue we should model?" Addison asked.

"We need to know the status of a trouble ticket at any given moment—the business has requested this feature, and it makes it easier to track several metrics. That implies we need an orchestrator so that we can query the state of the workflow."

"But you don't have to have an orchestrator for that—we can query any given service to see if it has handled a particular part of the workflow, or use stamp coupling," said Addison.

"That's right—this isn't a zero-sum game," said Logan. "It's possible that both or neither work just as well. We'll give both solutions credit in our updated table in Table 11-7."

TRADE-OFFS

Table 11-7. Updated trade-offs between orchestration and choreography for ticket workflow

Orchestration	Choreography
Workflow control	
State query	State query

"OK, what else?"

"Just one more that I can think of," Addison said. "Tickets can get canceled by the customer, and tickets can get reassigned because of expert availability, lost connections to the expert's mobile device, or expert delays at a customer site. Therefore, proper error handling is important. That means orchestration?"

"Yes, generally. Complex workflows must go somewhere, either in an orchestrator or scattered through services. It's nice to have a single place to consolidate error handling. And choreography definitely does not score well here, so we'll update our table in Table 11-8."

TRADE-OFFS

Table 11-8. Final trade-offs between orchestration and choreography for ticket workflow

Orchestration	Choreography
Workflow control	
State query	State query
Error handling	

"That looks pretty good. Any more?"

"Nothing that's not obvious," said Addison. "We'll write this up in an ADR; in case we think of any other issues, we can add them there."

> *ADR: Use Orchestration for Primary Ticket Workflow*
>
> *Context*
> *For the primary ticket workflow, the architecture must support easy tracking of lost or mistracked messages, excellent error handling, and the ability to track ticket status. Either an orchestration solution illustrated in Figure 11-16 or a choreography solution illustrated in Figure 11-15 will work.*
>
> *Decision*
> *We will use orchestration for the primary ticketing workflow.*
>
> *We modeled orchestration and choreography and arrived at the trade-offs in Table 11-8.*
>
> *Consequences*
> *Ticketing workflow might have scalability issues around a single orchestrator, which should be reconsidered if current scalability requirements change.*

Previous chapter

< 10. Distributed Data Access

Next chapter

12. Transactional Sagas >